

要求分析

1. 簡単なアプリの概要

本アプリは、通常のToDoリストに「タスク同士の関係を視覚的に把握する」機能を加えた、Canvas型のToDoアプリである。ユーザーはタスクをカードとしてキャンバス上に配置し、タスク同士を線でつなぐことで、親子関係、依存関係、作業の流れなどを表現できる。

一般的なToDoアプリは、期限や優先度によってタスクを一覧管理することに向いている。一方で、複数のタスクがどのようにつながっているか、どの作業を先に行う必要があるか、どの範囲まで作業を進めるべきかは把握しにくい。本アプリでは、Obsidian Canvasのような2次元の自由配置と接続線を使い、タスクの構造を見ながら管理できるようにする。

また、接続関係から算出される「深さ」によってタスクをフィルタリングできるようにする。本アプリにおける深さは、あらかじめ固定された階層ではなく、「ユーザーが選択した基準タスクから、接続線を何本たどれば到達できるか」を表す相対的な距離として定義する。例えば、選択中のタスクを深さ0、そこに直接つながるタスクを深さ1、さらにその先につながるタスクを深さ2として扱う。これにより、あるタスクを中心に関連作業だけを確認したい場合と、周辺の作業を広く確認したい場合を切り替えられる。

想定する利用者は、授業課題、個人開発、レポート作成、イベント準備など、複数の作業を分解して進めたい学生または個人ユーザーである。実装技術は生のHTML、CSS、JavaScriptとし、サーバーを用いずブラウザ上で動作するシンプルなアプリを目指す。

2. 競合分析

2.1 Obsidian Canvas

Obsidian Canvasは、ノート、添付ファイル、Webページなどを2次元空間に配置し、それらを線で接続できる視覚的なノート機能である。公式ヘルプでは、無限に近いキャンバス上に情報を配置し、線でつなぎ、関連するカードをグループ化できる機能として説明されている。また、接続線にはラベルや色を付けられ、キャンバスはJSON Canvas形式で保存される。

本アプリとの関連では、カードの自由配置、接続線、グループ化、ズーム・パン操作が参考になる。ただし、Obsidian Canvasはノート管理が中心であり、ToDoの完了状態、期限、優先度、深さによるタスク管理は主目的ではない。そのため、本アプリではCanvas型UIをToDo管理に特化させる点に差別化の余地がある。

参考: [Canvas - Obsidian Help](#)

2.2 Todoist

Todoistは、タスク管理に特化した代表的なToDoアプリである。タスク名、期限、プロジェクト、ラベル、優先度、作成日などを条件にフィルターを作成できる。公式ヘルプでは、フィルターを「特定条件に基づいたタスクのカスタムビュー」として説明しており、サブタスク、日付、優先度、ラベル、プロジェクトなどによる絞り込みができる。

本アプリとの関連では、タスクを単に追加・完了するだけでなく、条件に応じて見たいタスクだけを表示する設計が参考になる。特に、深さフィルターはTodoistのフィルター機能に相当する本アプリ独自の主要機能

と考えられる。一方で、Todoistはリスト中心であり、タスク間の関係を線で可視化する機能は主機能ではない。

参考: [Introduction to filters - Todoist Help](#)

2.3 Trello

Trelloは、ボード、リスト、カードを使ってタスクやプロジェクトを整理するカンバン型のアプリである。公式サイトでは、ボードを作成して作業を整理できること、Inbox、Boards、Plannerなどを使ってToDoやアイデア、責任範囲を管理できることが説明されている。

本アプリとの関連では、タスクをカードとして扱う点、視覚的に進捗や状態を把握する点が参考になる。Trelloは「未着手」「進行中」「完了」のような状態管理に強いが、カード同士を任意の線で接続し、階層や依存関係を表す用途には限定的である。本アプリでは、カンバン列ではなく自由なCanvas配置を採用することで、タスク間の関係性をより直接的に表現する。

参考: [Trello](#)

2.4 ClickUp

ClickUpは、タスク管理、チャット、ホワイトボード、ドキュメント、複数の表示形式を統合したプロジェクト管理ツールである。調査した情報では、リスト、ボード、カレンダー、ガントチャートなど複数のビューに加え、タスクの依存関係、ホワイトボード、ドキュメント連携などを持つとされている。

本アプリとの関連では、タスクを複数の見方で扱うこと、依存関係を明示することが参考になる。ただし、ClickUpは多機能なチーム向けツールであり、授業課題として作る個人向けアプリとしては機能過多である。本アプリでは、チーム管理や高度な自動化ではなく、タスク作成、接続、深さフィルター、保存といった基本機能に絞るべきである。

参考: [ClickUp - Wikipedia](#)

3. 競合分析から見た機能要求

3.1 タスク作成・編集機能

- ・ ユーザーはタスクカードを新規作成できる。
- ・ タスクカードには、タイトル、説明、完了状態を設定できる。
- ・ 必要に応じて、期限、優先度、タグを設定できる。
- ・ タスクカードは編集、削除できる。
- ・ 完了済みタスクは見た目で未完了タスクと区別できる。

3.2 Canvas表示機能

- ・ タスクはカードとしてキャンバス上に表示する。
- ・ ユーザーはカードをドラッグして任意の位置に配置できる。
- ・ キャンバスはタスク同士の関係が見やすい余白を持つ。
- ・ タスク数が増えても全体像を把握できるよう、可能であればズームまたは表示範囲の調整機能を用意する。
- ・ 画面上には、タスク一覧ではなくCanvasを主要な操作領域として表示する。

3.3 タスク接続機能

- ・ ユーザーは2つのタスクカードを線で接続できる。
- ・ 接続線は、親子関係または依存関係を表すものとして扱う。
- ・ 接続線は削除できる。
- ・ 同じ2つのタスク間に重複した接続が作られないようにする。
- ・ 循環関係があっても深さ計算が停止しないよう、同じタスクを重複して探索しない処理を行う。

3.4 深さ計算・深さフィルター機能

- ・ 深さは絶対的な階層ではなく、ユーザーが選択した基準タスクから接続線をたどったときの相対的な距離として扱う。
- ・ 基準タスクを深さ0とし、直接つながるタスクを深さ1、その先のタスクを深さ2以上として計算する。
- ・ 複数の経路で到達できるタスクは、最も短い経路の距離を深さとして扱う。
- ・ ユーザーは基準タスクと表示する最大深さを選択できる。
- ・ 深さフィルターの適用中は、表示対象外のタスクと、それに接続する線を非表示にする。

3.5 タスク検索・絞り込み機能

- ・ タスク名または説明文でキーワード検索できる。
- ・ 完了状態で表示を切り替えられる。
- ・ 優先度やタグを実装する場合、それらでも絞り込める。
- ・ 深さフィルターと他の絞り込み条件は同時に利用できる。

3.6 データ保存機能

- ・ 作成したタスク、位置、接続関係、完了状態、フィルター設定を保存できる。
- ・ 生のHTML、CSS、JavaScriptで完結させるため、まずはLocalStorageを使ったブラウザ内保存を想定する。
- ・ ページを再読み込みしても、前回の状態が復元される。
- ・ 必要に応じて、JSON形式でのエクスポート・インポート機能を検討する。

3.7 操作性に関する機能

- ・ タスク追加、接続、削除、完了切り替えが画面上で直感的に行える。
- ・ 接続モードと通常のカード移動モードを明確に切り替えられる。
- ・ 誤操作を防ぐため、削除時には確認または取り消し手段を用意する。
- ・ タスク数、表示中の深さ、表示中のタスク数が分かる簡単なステータス表示を用意する。

4. ToDoアプリとしての非機能要求

4.1 使いやすさ

- ・ 初めて使うユーザーでも、タスクの追加、移動、接続、完了、フィルター操作の入口が分かりやすいこと。
- ・ タスクカードは、タイトルや完了状態などの重要な情報が読み取りやすく、未完了タスクと完了済みタスクを見ただけで区別できること。
- ・ 接続モードでは、現在の操作状態や選択中のタスクが分かり、どのタスク同士を接続しようとしているか把握しやすいこと。
- ・ 深さフィルターでは、基準タスク、最大深さ、表示対象が分かり、フィルターを変更した結果がすぐ画面に反映されること。

- ・削除や接続解除などの取り消しにくい操作では、確認表示や取り消し手段によって誤操作を防げること。

4.2 応答性・性能

- ・タスク数が数十件程度であれば、ドラッグ操作やフィルター切り替えが遅延なく動作すること。
- ・深さ計算は、タスク追加、接続変更、フィルター変更時に短時間で完了すること。
- ・画面再描画は必要な範囲に抑え、操作中にちらつきが起きないこと。

4.3 信頼性

- ・ページを再読み込みしてもデータが失われないこと。
- ・不正な接続、存在しないタスクへの接続、循環関係などでアプリが停止しないこと。
- ・LocalStorageの読み込みに失敗した場合でも、初期状態でアプリを起動できること。

4.4 保守性

- ・HTML、CSS、JavaScriptの役割を分離すること。
- ・JavaScriptでは、タスク管理、描画、保存、フィルター処理を可能な限り関数単位で分けること。
- ・タスクデータの構造を明確にし、後から期限やタグなどの属性を追加しやすくすること。
- ・授業課題として説明しやすいよう、複雑すぎる外部ライブラリやビルド環境には依存しないこと。

4.5 互換性

- ・最新の主要ブラウザで動作すること。
- ・サーバーを必要とせず、HTMLファイルを開くだけで基本機能が使えること。
- ・画面幅が狭い環境でも、最低限タスクの閲覧と編集ができること。

4.6 アクセシビリティ

- ・ボタンや入力欄の意味が分かるラベルを付けること。
- ・色だけに依存せず、完了状態や選択状態が分かるようにすること。
- ・キーボード操作でも、基本的なタスク追加・編集ができるよう配慮すること。

4.7 セキュリティ・プライバシー

- ・ユーザー入力をHTMLとしてそのまま挿入せず、スクリプト実行につながる処理を避けること。
- ・JSONインポートを実装する場合、不正な形式のデータを読み込んでもアプリが壊れないよう検証すること。

4.8 授業課題としての実現可能性

- ・実装範囲は、生のHTML、CSS、JavaScriptで完成できる規模にすること。
- ・最小実装では、タスクカード、接続線、深さフィルター、LocalStorage保存を優先する。
- ・期限、タグ、JSONエクスポート、ズームなどは、基本機能が完成した後の追加機能として扱う。
- ・画面の見た目よりも、タスク間の接続と深さフィルターという独自性が伝わることを優先する。